IIMT SDK API Description

1.IIMT SDK API Description

SDK & API Instructions
It is the SDK package released with the update version and the c++ interface API provided. Users can carry out simple secondary development according to the interface instructions to control the robot’s state and motion form PC side
What is SDK and API interface?
SDK(software development kit),It is a conllection of development tools for building application software,such as specific software packages,software architectures,hardware platforms,and operatiing systems,whic are used only by soft engineers.
API(application programming interface),It is the interface for user programming,it is the pre-defined functions which aim to provide applications and developers with the ability to access a set of routines based on a piece of software or hardware.For secondary developers,just need to focus on the functionality of interface,not the internal imploementation details.
Communication method
The SDK interface and controller communicate through TCP/IP, and as a TCP server, the controller defaults to listening on port 29999.

2.SDK & API Sheet

serial number	Interface Name	Interface Function	Remark
1	API_HelloWorld	Test interface	
2	API_Login	Login, establish connection	
3	API_Logout	Log out, release connection	
4	API_SetDebugMode	Set whether to debug mode or not	
5	API_GetDebugMode	Get whether to debug mode or not	
6	API_GetErrorInfo	Get error message description	
7	API_PowerOn	Power on	
8	API_ReleaseBrake	Release the brake	
9	API_PowerOff	Power off	
10	API_Play	Start	
11	API_Stop	Stop	
12	API_Pause	Pause	

serial number	Interface Name	Interface Function	Remark
13	API_LoadProgram	Load program	
14	API_LoadInstallation	Load installation configuration	
15	API_Shutdown	Shut down controller	
16	API_Running	Running status query	
17	API_IsProgramSaved	Query if the program has been saved	
18	API_IsInRemoteControl	Query if it is remote control	
19	API_Popup	Popup	
20	API_ClosePopup	Close the popup	
21	API_AddToLog	Add log messages to log history	
22	API_GetControllerVersion	Get version information of software installed on the robot	
23	API_GetVersion	Get version number	
24	API_GetRobotModel	Get robot model	
25	API_GetSerialNumber	Get serial number	
26	API_GetRobotMode	Get robot mode	
27	API_GetLoadedProgram	Get which program is loaded	
28	API_GetSafetyStatus	Get Safety status	
29	API_GetProgramState	Get program state	
30	API_GetOperationalMode	Get operational mode	
31	API_SetOperationalMode	Set operational mode	
32	API_ClearOperationalMode	Clear operational mode	
33	API_FreeDriveMode	Free drag mode	
34	API_EndFreeDriveMode	End free drag mode	
35	API_MannulTeachStart	Mannul teach start	
36	API_API_ManualTeachEnd	Mannul teach end	
37	API_Servoj	Joint space servo motion	Need to establish a separate connection,achieve lower latency
38	API_Servoc	Cartesian space servo motion	Need to establish a separate connection,achieve lower latency

serial number	Interface Name	Interface Function	Remark
39	API_Movej	Joint motion	
40	API_Movel	Linear motion	
41	API_Movepc	Circle motion	
42	API_GetDI	Get digital input status	
43	API_GetDO	Get digital output status	
44	API_GetAllDI	Get all digital input status at once	
45	API_GetAllDO	Get all digital output status at once	
46	API_SetDO	Set the digital output status	
47	API_SetAO	Set the analog output value	
48	API_GetRobotStatus	Get robot status	
49	API_GetRobotJointRad	Get joint position radian	
50	API_GetRobotJointAngle	Get joint position angle	
51	API_GetRobotTcpPosition	Get TCP position	
52	API_GetJointArriveStatus	Get joint arrive status	
53	API_GetTcpArriveStatus	Get TCP arrive status	
54	API_GetRunSpeed	Get running speed	
55	API_SetRunSpeed	Set running speed	
56	API_ExecuteScript	Remote script command execution	
57	API_GetVariableValue	Get variable value	
58	API_SetOffset	Set trajectory offset	
59	API_GetOffset	Get trajectory offset	
60	API_SendFile	Send files to the teaching pendant	
61	API_ExecuteFile	Remote execution of Lua script files	
62	API_GetCollisionLevel	Get collision level	
63	API_SetCollisionLevel	Set collision level	
64	API_OpenSerialPort	Open serialport	
65	API_SendRtuCmd	Send modbu rtu commands	
66	API_ModbusRtuRW	Customize modbus rtu commands for reading and writing	

serial number	Interface Name	Interface Function	Remark
67	API_CloseSerialPort	Close serialport	
68	API_TcpPositionCalibration	Calibration of tool end position	
69	API_TcpDirectionCalibration	Calibration of tool end direction	
70	API_GetDragButtonStatus	Get the status of the drag button at the end	
71	API_GetPointSetButtonStatus	Get the status of point setting button a the end	
72	API_GetRobotErrorCode	Get robot error code	
73	API_InverseKin	Inverse kinematics	
74	API_ForwardKin	Forward kinematics	
75	API_PointDist	Distance between two points	
76	API_PoseAdd	Pose addition	
77	API_PoseInv	Get the inverse of a pose	
78	API_PoseSub	Pose subtraction	
79	API_PoseTrans	Pose transformation	
80	API_GetRobotHomeJointRad	Get the radian of the robot’s zero-position joint	
81	API_ResetRobotError	Robot error reset	
82	API_GetRobotRunTime	Get the robot’s runtime	
83	API_GetPowerOffDelayTime	Get the power-off delay time of the robot	
84	API_EmergencyStop	Emergency stop of the robot	
85	API_ClearUnfinishedTrajectory	Clear unfinished trajectory	
86	API_GetPointsInProgram	Get waypoints information in the program	
87	API_GetFirmwareVersion	Get firmware version	
88	API_SetMountedPose	Set the installation method	
89	API_SetMountedAngle	Set the installation angle	
90	API_ModbusRtuRWEndTool	End tool Modbus RTU read and write	

- Remark：

3.SDK API Return Value Description

Return Samples
• Success：TRUE
• Fail：FALSE
Return parameter description

Parameter Name	Type	Description
TRUE	int	1
FALSE	int	0

Remark
•

4.SDK API Interface Description

4.1.Testing Interface

Brief Description
• Testing Interface
Interface Definition
• void API_HelloWorld()
Remark
• This interface is used by secondary developers for testing. It has nothing to do with robots. Verify whether the user's library is loaded and the interface is exported correctly. After everything is successful, calling this interface will print "Hello World!"

4.2.Login, establish connection，all the basics of robot operation

Brief Description
• Initialize and establish connection
Interface Definition
• BOOL API_Login(char* ip, int port,JTRHD& handle)
Parameter

Parameter Name	Required	Type	Description
ip	Yes	char*	IP address
port	Yes	int	port
handle	Yes	JTRHD	Login handle, input, output

Remark
•

4.3.Log out, disconnect, and cannot operate the robot again after disconnection

Brief Description
• Log out, disconnect
Interface Definition
• BOOL API_Logout(JTRHD handle)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input

Remark
•

4.4.Set whether to debug mode or not

Brief Description
• When set to debug mode, debug information will be printed
Interface Definition
• BOOL API_SetDebugMode(JTRHD handle,const BOOL is_debug)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
is_debug	Yes	BOOL	1:Yes 0:No.input

Remark
•

4.5.Get whether to debug mode or not

Brief Description
• Get whether the current mode is debugging mode
Interface Definition
• BOOL API_GetDebugMode(JTRHD handle,const BOOL& is_debug)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
is_debug	Yes	BOOL	1：Yes 0：No. input, output

Remark
•

4.6.Get error message description

Brief Description
• Get the error description of the last API call error
Interface Definition
• BOOL API_GetErrorInfo(JTRHD handle,const ErrorInfo* info)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
info	Yes	ErrorInfo	input, output

Remark
•

4.7.Power on

Brief Description
• Power on
Interface Definition
• BOOL API_PowerOn(JTRHD handle)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input

Remark
• If the robot has already been powered on and is powered on repeatedly, the interface returns a failure. Can be used in conjunction with pattern queries or power off interfaces.

4.8.Release the brake

Brief Description
• Release the brake
Interface Definition
• BOOL API_ReleaseBrake(JTRHD handle)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input

Remark
•The robot brake is released. When the machine is powered on, this function is sent through state feedback to release the brake and activate the robot.

4.9.Power Off

Brief Description
• Power off
Interface Definition
• BOOL API_PowerOff(JTRHD handle)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input

Remark
• Robot body powered off

4.10.Start

Brief Description
• Start the robot program to run
Interface Definition
• BOOL API_Play(JTRHD handle)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input

Remark
•

4.11.Stop

Brief Description
• Stop the robot program from running
Interface Definition
• BOOL API_Stop(JTRHD handle)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input

Remark
•

4.12.Pause

Brief Description
• Pause the robot program
Interface Definition
• BOOL API_Pause(JTRHD handle)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input

Remark
•

4.13.Load Program

Brief Description
• Open existing programs remotely
Interface Definition
• BOOL API_LoadProgram(JTRHD handle,const char* program_file_name)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
program_file_name	Yes	char*	program name，input

Remark
•

4.14.Load installation configuration

Brief Description
• Remote load installation configuration
Interface Definition
• BOOL API_LoadInstallation(JTRHD handle,const char* installation_file_name)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
installation_file_name	Yes	char*	Installation configuration file name，input

Remark
•

4.15.Shut down controller

Brief Description
• Shut down controller
Interface Definition
• BOOL API_Shutdown(JTRHD handle)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input

Remark
•

4.16.Running status query

Brief Description
• Running status query
Interface Definition
• BOOL API_Running(JTRHD handle,ProgramRunning* program_running)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
program_running	Yes	ProgramRunning*	Description of running status，input, output

Remark
•

4.17.Query if the program has been saved

Brief Description
• Query if the program has been saved
Interface Definition
• BOOL API_IsProgramSaved(JTRHD handle,ProgramSaved* program_saved)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
program_saved	Yes	ProgramSaved*	Whether to save the description，input, outp

Remark
•

4.18.Query if it is remote control

Brief Description
• Query if it is remote control
Interface Definition
• BOOL API_IsInRemoteControl(JTRHD handle,InRemoteControl* in_remote_control)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
in_remote_control	Yes	InRemoteControl*	Description of remote control，input, output

Remark
•

4.19.Popup

Brief Description
•
Interface Definition
• BOOL API_Popup(JTRHD handle,const char* popup_text)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
popup_text	Yes	char*	Pop up message，input

Remark
•

4.20.Close the popup

Brief Description
•
Interface Definition
• BOOL API_ClosePopup(JTRHD handle)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input

Remark
•

4.21.Add log messages to log history

Brief Description
•
Interface Definition

• BOOL API_AddToLog(JTRHD handle,const char* log_text)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
log_text	Yes	char*	Log messages，input

Remark
•

4.22.Get version information of software installed on the robot

Brief Description
•
Interface Definition
• BOOL API_GetControllerVersion(JTRHD handle,ControllerVersion* controller_version)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
controller_version	Yes	ControllerVersion*	Version information description，input, output

Remark
•

4.23.Get version number

Brief Description
•
Interface Definition
• BOOL API_GetVersion(JTRHD handle,Version* version)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
version	Yes	Version*	Version number information description，input, output

Remark
•

4.24.Get robot model

Brief Description
• iimt3、iimt5、iimt10、iimt16、iimt20 and so on
Interface Definition
• BOOL API_GetRobotModel(JTRHD handle,RobotModel* robot_model)
Parameter

Parameter Name	Required	Type	Description

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
robot_model	Yes	RobotModel*	Robot model information description，input, output

Remark
•

4.25.Get serial number

Brief Description
•
Interface Definition
• BOOL API_GetSerialNumber(JTRHD handle,SerialNumber* serial_number)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
serial_number	Yes	SerialNumber*	Serial number information description，input, output

Remark
•

4.26.Get robot mode

Brief Description
•
Interface Definition
• BOOL API_GetRobotMode(JTRHD handle,RobotMode* robot_mode)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
robot_mode	Yes	RobotMode*	Robot mode information description，input, output

Remark
•

4.27.Get which program is loaded

Brief Description
•
Interface Definition
• BOOL API_GetLoadedProgram(JTRHD handle,LoadedProgram* loaded_program)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input

Parameter Name	Required	Type	Description
loaded_program	Yes	LoadedProgram*	Information description of the loaded program，input, output

Remark

•

“Loaded program: path toloaded program file” OR “No program loaded”

4.28.Get Safety status

Brief Description

•

Interface Definition

• BOOL API_GetSafetyStatus(JTRHD handle,SafetyStatus* safety_status)

Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
safety_status	Yes	SafetyStatus*	Description of Safety Status Information，input, output

Remark

•

4.29.Get program state

Brief Description

•

Interface Definition

• BOOL API_GetProgramState(JTRHD handle,ProgramState* program_state)

Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
program_state	Yes	ProgramState*	Description of program status information，input, output

Remark

•

4.30.Get operational mode

Brief Description

•

Interface Definition

• BOOL API_GetOperationalMode(JTRHD handle,OperationalMode* operational_mode)

Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input

Parameter Name	Required	Type	Description
operational_mode	Yes	OperationalMode*	Operation mode information description，input, output

Remark
•

4.31.Set operational mode

Brief Description
•
Interface Definition
• BOOL API_SetOperationalMode(JTRHD handle,const char* operational_mode)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
operational_mode	Yes	char*	Operational mode，manual/automatic，input

Remark
•

4.32.Clear operational mode

Brief Description
•
Interface Definition
• BOOL API_ClearOperationalMode(JTRHD handle)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input

Remark
•

4.33.Free drag mode

Brief Description
• Free drag mode
Interface Definition
• BOOL API_FreeDriveMode(JTRHD handle)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input

Remark
•

4.34.End free drag mode

Brief Description
• End free drag mode
Interface Definition
• BOOL API_EndFreeDriveMode(JTRHD handle)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input

Remark
•

4.35.Mannul teach start

Brief Description
• Mannul teach start
Interface Definition
• BOOL API_ManualTeachStart(JTRHD handle,int num,int direction,float step_size = 0.0)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
num	Yes	int	input, See remarks
direction	Yes	int	input, See remarks
step_size	Yes	float	input, Step size, unit: millimeter (mm), valid in cartesian space

Remark
• The manual teaching function instruction is equivalent to single joint and axis operations in the moving tab of the teaching pendant. The reference is as follows:
num
range 1~12
direction
0:reverse movement 1: positive movement

1~6
Joint number, joint movement
7~9
Cartesian coordinate movement x , y , z
Axis reverse movement
Axis positive movement
10~12
Cartesian rotation Rx , Ry , Rz
Reverse posture transformation
Positive posture transformation
Reference examples: 1 API_ManualTeachStart(handle,1,0) // The first joint runs in reverse 2 API_ManualTeachStart(handle,7,1) //X-axis positive movement 3 API_ManualTeachStart(handle,10,0) //Rx reverse movement

4.36.End mannul teach

Brief Description
• End mannul teach

Interface Definition
• BOOL API_ManualTeachEnd(JTRHD handle,int num,int direction)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
num	Yes	int	See remarks
direction	Yes	int	See remarks

Remark
• The manual teaching end instruction is used after the teaching start instruction to end the teaching. Reference example: 1 API_ManualTeachEnd(handle,1,0) // The first joint has completed reverse operation 2 API_ManualTeachEnd(handle,7,1) //End of positive X-axis movement 3 API_ManualTeachEnd(handle,10,0) //Rx reverse movement ends

4.37.Joint space servo motion

Brief Description
•
Interface Definition
• BOOL API_Servoj(JTRHD handle,ServojParam* parameter)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
parameter	Yes	ServojParam*	input

typedef struct
{
double q[6]; //Target point joint parameters, format: {0.0,1.57,0.0,-1.57,-1.57,0} ,unit: rad
float a;
float v;
float t;
float lookahead_time;
float gain;
}ServojParam;

Remark
• Need to establish a separate connection, achieve lower latency

4.38.Cartesian space servo motion

Brief Description
•
Interface Definition
• BOOL API_Servoc(JTRHD handle,ServocParam* parameter)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
parameter	Yes	ServocParam*	input

typedef struct
{

double p[6]; //TCP parameters, x, y, z in units of m, rx, ry, rz in units of rad
float a;
float v;
float t;
float lookahead_time;
}ServocParam;

Remark
• Need to establish a separate connection, achieve lower latency

4.39.Joint motion

Brief Description
•
Interface Definition
• BOOL API_Movej(JTRHD handle,MovejParam* parameter,bool is_block=true,int timeout=30)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
parameter	Yes	MovejParam*	input
is_block	Yes	int	true: block，false: non-block
timeout	Yes	int	timeout: The default value is 30s, which is valid when is-block is true

typedef struct
{
double q[6]; //Target point joint parameters, format: {0.0,1.57,0.0,-1.57,-1.57,0} ,unit: rad
float v; //Speed percentage,range [1,100]
float a; //Joint speed,rad/s
float j; //Joint acceleration,rad/s^2
float r; //Trajectory blending radius,mm
float t; //Trajectory planning time, system retains parameter interface, can be left blank
}MovejParam;

Remark
•

4.40. Linear motion

Brief Description
• Linear motion
Interface Definition
• BOOL API_Movel(JTRHD handle,MovelParam* parameter,bool is_block=true,int timeout=30)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
parameter	Yes	MovelParam*	input
is_block	Yes	int	true: block，false: non-block

Parameter Name	Required	Type	Description
timeout	Yes	int	timeout: The default value is 30s, which is valid when is-block is true

typedef struct
{
double p[6]; //TCP parameters, x, y, z in units of m, rx, ry, rz in units of rad
float v; //speed
float tool_v; //Tool speed
float tool_a; //Tool acceleration
float r; //blending radius,mm
}MovelParam;

Remark
•

4.41. Circle motion

Brief Description
• Circle motion
Interface Definition
• BOOL API_Movepc(JTRHD handle,MovepcParam* parameter,bool is_block=false,int timeout=0)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
parameter	Yes	MovepParam*	input
is_block	Yes	int	true: block，false: non-block
timeout	Yes	int	timeout: The default value is 0s, which is valid when is-block is true

typedef struct
{
double passing_point[6]; //Passing through points, TCP parameters, TCP parameters, x, y, z in units of m, rx, ry, rz in units of rad
double end_point[6]; //End point, TCP parameters, TCP parameters, x, y, z in units of m, rx, ry, rz in units of rad
float tool_v; //Tool speed
float tool_a; //Tool acceleration
int circle; //Circle or Arc 1: Whole Circle 0: Arc
int count; //Circle times
float r; //blend radius,mm
int mode; //Mode 1: Fixed 0: Unconstrained
float reserve; //Reserved field
}MovepcParam;

Remark
• Is-block uses default value of false

Remark
•

4.42.Get digital input status

Brief Description
• Get digital input status
Interface Definition
• BOOL API_GetDI(JTRHD handle,int num,IoStatus& status)

Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
num	Yes	int	0~15
status	Yes	IoStatus	input, output

typedef enum
{
IO_STATUS_LOW = 0,
IO_STATUS_HIGH
}IoStatus;
Remark
•

4.43.Get digital output status

Brief Description
•Get digital output status
Interface Definition
• BOOL API_GetDO(JTRHD handle,int num,IoStatus& status)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
num	Yes	int	0~15
status	Yes	IoStatus	input, output

typedef enum
{
IO_STATUS_LOW = 0,
IO_STATUS_HIGH
}IoStatus;
Remark
•

4.44.Get all digital input status at once

Brief Description
•
Interface Definition
•BOOL API_GetAllDI(JTRHD handle,AllIoStatus* status)

Parameter

Parameter Name	Required	Type	Description

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
num	Yes	int	0~15
status	Yes	AllIoStatus	input, output

Remark
•

4.45.Get all digital output status at once

Brief Description
•
Interface Definition
•BOOL API_GetAllDO(JTRHD handle,AllIoStatus* status)

Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
num	Yes	int	0~15
status	Yes	AllIoStatus	input, output

Remark
•

4.46.Set digital output status

Brief Description
•
Interface Definition
• BOOL API_SetDO(JTRHD handle,int num,IoStatus status)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
num	Yes	int	0~15
status	Yes	IoStatus	input

typedef enum
{
IO_STATUS_LOW = 0,
IO_STATUS_HIGH
}IoStatus;

Remark
•

4.47.Set analog output value

Brief Description
•
Interface Definition

• BOOL API_SetAO(JTRHD handle,int num,float value)
Parameter

Parameter Name	Required	Type	Description
handle	yes	JTRHD	Login handle，input
num	yes	int	range:[0~3]
value	yes	float	input,when num is 0 or 1,range[0,10],unit(V),when num is 2 or 3,range[4,20],unit(mA),All rounded to two decimal places

Remark
•

4.48.Get robot status

Brief Description
•
Interface Definition
• BOOL API_GetRobotStatus(JTRHD handle,RobotStatus& status)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
num	Yes	int	0~15
status	Yes	RobotStatus	input, output

typedef enum
{
ROBOT_POWEROFF, //0
ROBOT_POWEROFF_2_POWERON,//1
ROBOT_POWERON,//2
ROBOT_POWERON_2_STANBY,//3
ROBOT_STANDBY,//4
ROBOT_STANDBY_2_MAUNTEACH,//5
ROBOT_MAUNTEACH,//6
ROBOT_MANUTEACH_2_MANUTEACHEND,//7
ROBOT_MAUNTEACH_END,//8
ROBOT_SINGLETARGETMOVE,//9
ROBOT_PAUSE,
ROBOT_STANBY_2_AUTORUN,//11
ROBOT_AUTORUN,//—-12
ROBOT_FREEDRIVE,//13
ROBOT_FREEDRIVE_END,//14
ROBOT_STANDBY_2_RUNTARGET,//15
ROBOT_RUNTARGET,//16
ROBOT_STOP,//17
ROBOT_EMERGENCY,//18
ROBOT_ERROR,//19
ROBOT_LOCAL_REMOTE,//20

ROBOT_COLLISION,
}RobotStatus;
Remark
•

4.49.Get joint position radian

Brief Description
•
Interface Definition
• BOOL API_GetRobotJointRad(JTRHD handle,JointRad* joint_rad)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
joint_rad	Yes	JointRad*	input, output

typedef struct
{
double robotRad[6]; //The joint radian values of the robotic arm
}JointRad;

Remark
•

4.50.Get joint position angle

Brief Description
•
Interface Definition
• BOOL API_GetRobotJointAngle(JTRHD handle,JointAngle* joint_angle)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
position	Yes	JointPosition	input, output

typedef struct
{
double robotAngle[6]; //The joint angle values of the robotic arm
}JointAngle;

Remark
•

4.51.Get tool position

Brief Description
•
Interface Definition
• BOOL API_GetRobotTcpPosition(JTRHD handle,TcpPosition* position)
Parameter

Parameter Name	Required	Type	Description

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
position	Yes	TcpPosition*	input, output

typedef struct
{
double robotTcp[6]; //The position and posture of the end effector of the robotic arm
}TcpPosition;

Remark
•

4.52.Get joint arrive status

Brief Description
•
Interface Definition
• BOOL API_GetJointArriveStatus(JTRHD handle,ArriveStatus& status,JointRad* joint_rad)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
status	Yes	ArriveStatus	input, output
joint_rad	Yes	JointRad*	input

typedef enum
{
ARRIVE_STATUS_ARRIVED = 0,
ARRIVE_STATUS_NOT_ARRIVED,
ARRIVE_STATUS_ERROR
}ArriveStatus;

typedef struct
{
double robotRad[6]; //The joint radian values of the robotic arm
}JointRad;

Remark
•

4.53.Get tcp arrive status

Brief Description
•
Interface Definition
• API_GetTcpArriveStatus(JTRHD handle,ArriveStatus& status,TcpPosition* position)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
status	Yes	ArriveStatus	input, output
position	Yes	TcpPosition*	input

```
typedef enum
{
ARRIVE_STATUS_ARRIVED = 0,
ARRIVE_STATUS_NOT_ARRIVED,
ARRIVE_STATUS_ERROR
}ArriveStatus;

typedef struct
{
double robotTcp[6]; //The position and posture of the end effector of the robotic arm
}TcpPosition;
```

Remark
•

4.54.Get running speed

Brief Description
•
Interface Definition
• BOOL API_GetRunSpeed(JTRHD handle,int& speed)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
speed	Yes	int	input, output

Remark
•What is obtained here is not the actual speed, but a percentage value. Range: (0 ~ 100)

4.55.Set running speed

Brief Description
•
Interface Definition
• BOOL API_SetRunSpeed(JTRHD handle,int speed)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
speed	Yes	int	input

Remark
•What is obtained here is not the actual speed, but a percentage value. Range: (0 ~ 100)

4.56.Script command execution

Brief Description
• Remote execution of single or multiple line scripts through SDK interface, separated by “;” between multiple line scripts, without line break symbol “\ n”.
Interface Definition
• BOOL API_ExecuteScript(JTRHD handle,const char* script_content)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
script_content	Yes	char*	Script content executed,input

Remark
•Due to the timing of script execution being determined by the content of the script, this interface returns immediately. Users need combine other interface functions such as program state or reach state to determine whether the script has been successfully executed.

4.57.Get variable value

Brief Description
• Retrieve the variable value of a certain variable name in the program
Interface Definition
• BOOL API_GetVariableValue(JTRHD handle,const char *var_name, VariableValue* value)

Parameter
Parameter Name	Required	Type	Description
:——	:———-	:—-
handle	Yes	JTRHD	Login handle, input
var_name	Yes	char	*input，Name of variable*
value	*Yes*	*VariableValue*	input/output，Description of the value of a variable	

Remark
ypedef struct
{
char description[512];
}VariableValue;

4.58.Set trajectory offset

Brief Description
•
Interface Definition
•BOOL API_SetOffset(JTRHD handle,TrajectoryOffset offset)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
offset	Yes	TrajectoryOffset	input

typedef struct
{
double x_offset; //m
double y_offset; //m
double z_offset; //m
double rx_offset; //rad
double ry_offset; //rad
double rz_offset; //rad
}TrajectoryOffset;

Remark
•

4.59.Get trajectory offset

Brief Description

•

Interface Definition

•BOOL API_GetOffset(JTRHD handle,TrajectoryOffset* offset)

Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
offset	Yes	TrajectoryOffset*	input, output

typedef struct
{
double x_offset; //m
double y_offset; //m
double z_offset; //m
double rx_offset; //rad
double ry_offset; //rad
double rz_offset; //rad
}TrajectoryOffset;

Remark

•

4.60.Send files to the teaching pendant

Brief Description

•

Interface Definition

•BOOL API_SendFile(JTRHD handle,const char *local_absolute_path_file_name,const char* save_path,const char* save_file_name)

Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
local_absolute_path_file_name	Yes	char*	input，Local file name (absolute path)
save_path	Yes	char*	Save file path on the teaching pendant, input
save_file_name	Yes	char*	The file name presented on the teaching pendant, input

Remark

•

4.61.Remote execution of Lua script files

Brief Description

•

Interface Definition

•BOOL API_ExecuteFile(JTRHD handle,const char* remote_absolute_path_file_name)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle, input
remote_absolute_path_file_name	Yes	char*	Remote (on teaching pendant) absolute path file name, input

Remark
•Due to the timing of script file execution being determined by the content of the script, this interface returns immediately. Users need combine other interface functions such as program state or reach state to determine whether the script file has been successfully executed.

4.62.Get collision level

Brief Description
•
Interface Definition
•BOOL API_GetCollisionLevel(JTRHD handle,CollisionLevel& level)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle，input
level	Yes	CollisionLevel	level collision level，range[0,5],0:close collision。collision threshold 1:10N,2:20N,3:30,4:40N,5:50N，input,output

Remark
typedef enum
{
COLLISION_LEVEL_ZERO = 0,
COLLISION_LEVEL_ONE,
COLLISION_LEVEL_TWO,
COLLISION_LEVEL_THREE,
COLLISION_LEVEL_FOUR,
COLLISION_LEVEL_FIVE,
}CollisionLevel;

4.63.Set collision level

Brief Description
•
Interface Definition
•BOOL API_SetCollisionLevel(JTRHD handle,const CollisionLevel level)
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle，input

Parameter Name	Required	Type	Description
level	Yes	CollisionLevel	level collision level，range[0,5],0:Close collision。Collision threshold 1:10N,2:20N,3:30,4:40N,5:50N，input

Remark

•

4.64.Open serialport

Brief Description

Open serialport，Prepare modbus communication

Interface Definition

• BOOL API_OpenSerialPort(JTRHD handle,IIMT_SerialPortParameters* param);

Parameter

Parameter Name	Required	Type	Description
handle	yes	JTRHD	Login handle，input
param	yes	UART_Params*	char SerialPortName[32] write“ ttyUSB0” or “ ttyUSB1”and so on,Other see remark

Remark

```
typedef struct {
char SerialPortName[32];
IIMT_BaudRate BaudRate;
IIMT_DataBits DataBits;
IIMT_Parity Parity;
IIMT_StopBits StopBits;
```

} UART_Params;

```
enum IIMT_BaudRate {
Baud1200 = 1200,
Baud2400 = 2400,
Baud4800 = 4800,
Baud9600 = 9600,
Baud19200 = 19200,
Baud38400 = 38400,
Baud57600 = 57600,
Baud115200 = 115200,
UnknownBaud = -1
```

};

```
enum IIMT_DataBits {
Data5 = 5,
```

```
Data6 = 6,
Data7 = 7,
Data8 = 8,
UnknownDataBits = -1
```

};

```
enum IIMT_Parity {
NoParity = 0,
EvenParity = 2,
OddParity = 3,
SpaceParity = 4,
MarkParity = 5,
UnknownParity = -1
```

};

```
enum IIMT_StopBits {
OneStop = 1,
OneAndHalfStop = 3,
TwoStop = 2,
UnknownStopBits = -1
```

};

4.65.Send modbus rtu command

Brief Description
Send modbus rtu request command，and obtain a response
Interface Definition
• BOOL API_SendRtuCmd(JTRHD handle,const ModbusRTU_Frame *req,ModbusRTU_Frame* rsp,int timeout=2);
Parameter

Parameter Name	Required	Type	Description
handle	yes	JTRHD	Login handle，input
req	yes	ModbusRTU_Frame*	request
rsp	yes	ModbusRTU_Frame*	response
timeout	yes	int	timeout，default 2s

Remark
typedef struct {
unsigned char data[256]; // The message data (up to 255 bytes) is binary data, with each byte ranging from 0x00 to 0xFF
unsigned short data_len; // Actual message length, actual number of bytes in binary data
} ModbusRTU_Frame;

4.66.Customize modbus rtu commands for reading and writing

Brief Description
Send custom modbus rtu request command，and obtain a response
Interface Definition
• BOOL API_SendRtuCmd(JTRHD handle,const ModbusRTU_Request *req,ModbusRTU_Frame* rsp,int timeout=2);
Parameter

Parameter Name	Required	Type	Description
handle	yes	JTRHD	Login handle，input
req	yes	ModbusRTU_Request*	request
rsp	yes	ModbusRTU_Frame*	response
timeout	yes	int	timeout，default 2s

Remark
typedef struct
{
unsigned char slave_addr;
unsigned char func_code;
unsigned short start_addr;
unsigned short data_number; // Data quantity (number of registers/coils)
unsigned char data[250]; // The message data (up to 255 bytes) is binary data, with each byte ranging from 0x00 to 0xFF
unsigned char data_len; //Actual message length, actual number of bytes in binary data
}ModbusRTU_Request;

4.67.Close serialport

Brief Description
Close serialport
Interface Definition
•BOOL API_CloseSerialPort(JTRHD handle);
Parameter

Parameter Name	Required	Type	Description
handle	yes	JTRHD	Login handle，input

Remark

4.68.Calibration of tool end position

Brief Description
Calibration of tool end position
Interface Definition
•BOOL API_TcpPositionCalibration(JTRHD handle,const char* points)
Parameter

Parameter Name	Required	Type	Description
handle	yes	JTRHD	Login handle，input
points	yes	const char*	The information of multiple points，input，See remark

Remark
•points: The information of multiple points (joint position radians) segmented by semicolons (;) is in the form of xxx, xxx, xxx, xxx, xxx; xxx,xxx,xxx,xxx,xxx,xxx; … can only have 4 or 6 points

4.69.Calibration of tool end direction

Brief Description
Calibration of tool end direction

Interface Definition
•BOOL API_TcpDirectionCalibration(JTRHD handle,const char* points)
Parameter

Parameter Name	Required	Type	Description
handle	yes	JTRHD	Login handle，input
points	yes	const char*	The information of multiple points，input，See remark

Remark
•points: The information of multiple points (joint position radians) segmented by semicolons (;) is in the form of xxx, xxx, xxx, xxx, xxx; xxx,xxx,xxx,xxx,xxx,xxx; … can only have 3 points

4.70.Get the status of the drag button at the end

Brief Description
Get the status of the drag button at the end
Interface Definition
•BOOL API_GetDragButtonStatus(JTRHD handle,ToolButtonStatus& status)
Parameter

Parameter Name	Required	Type	Description
handle	yes	JTRHD	Login handle，input
status	yes	ToolButtonStatus	Button status information, input/output, See remark

Remark
typedef enum
{
BUTTON_STATUS_ERROR = -1, //The status is unavailable, such as the robot not being powered on and releasing the brake
BUTTON_STATUS_PRESS_DOWN = 0,//Press down the button
BUTTON_STATUS_RELEASE //Release the button
}ToolButtonStatus;

4.71.Get the status of point setting button a the end

Brief Description
Get the status of point setting button a the end
Interface Definition
•BOOL API_GetPointSetButtonStatus(JTRHD handle,ToolButtonStatus& status)
Parameter

Parameter Name	Required	Type	Description
handle	yes	JTRHD	Login handle，input
status	yes	ToolButtonStatus	Button status information, input/output, See remark

Remark
typedef enum
{
BUTTON_STATUS_ERROR = -1, //The status is unavailable, such as the robot not being powered on and releasing the

brake
BUTTON_STATUS_PRESS_DOWN = 0,//Press down the button
BUTTON_STATUS_RELEASE //Release the button
}ToolButtonStatus;

4.72.Get robot error code

Brief Description
Get robot error code
Interface Definition
•API_GetRobotErrorCode(JTRHD handle,int& error_code);
Parameter

Parameter Name	Required	Type	Description
handle	yes	JTRHD	Login handle，input
error_code	yes	int	error，input/output，See remark

Remark
0:normal
1: Robot joint speed too fast
2: Robot servo stalling
3: Robot hardware overcurrent
4: Current threshold exceeds the limit
5: Robot servo overheating
6: Robot grating pulse loss
7: Robot rotor positioning failure
8: Magnetic encoder error
9: Magnetic encoder over turn protection
10: Undervoltage protection
11: Robot relay failed to open
12: Hardware error
13: Magnetic and optical encoder position difference too large error
14: Magnetic encoder abnormal over turn error
15: Communication failure
16: Position out of tolerance protection
17: There is a jump point in the data sent by the upper computer
19: Robot body connection failed
20: Planning error
21: Exceeding the limit
22: Strange speed
40: Over temperature

4.73.Inverse kinematics

Brief Description
•
Interface Definition
•BOOL API_InverseKin(JTRHD handle,const TcpPosition *tcp_pose,const JointRad* ref_joint_rad,JointRad* joint_rad);
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle，input

Parameter Name	Required	Type	Description
tcp_pose	Yes	TcpPosition*	Cartesian spatial pose(unit:x/y/z (m),rx/ry/rz (radians))，input
ref_joint_rad	Yes	JointRad*	Reference joint space position for inverse solution calculation (unit: radians), input
joint_rad	Yes	JointRad*	Joint spatial position calculation result (unit: radians), input/output

Remark
typedef struct
{
double robotRad[6]; //Joint radian
}JointRad;

typedef struct
{
double robotTcp[6]; //The position and posture of the end effector of the robotic arm,Corresponding to x, y, z, rx, ry, rz respectively
}TcpPosition;

4.74.Formard kinematics

Brief Description
•
Interface Definition
•BOOL API_ForwardKin(JTRHD handle,const JointRad *joint_rad,TcpPosition* result);
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle，input
joint_rad	Yes	JointRad*	joint space position (unit: radians), input
result	Yes	TcpPosition*	calculation result(unit:x/y/z (m),rx/ry/rz (rad))), input/output

Remark
typedef struct
{
double robotRad[6]; //Joint radian
}JointRad;

typedef struct
{
double robotTcp[6]; //The position and posture of the end effector of the robotic arm,Corresponding to x, y, z, rx, ry, rz respectively
}TcpPosition;

4.75.Distance between two points

Brief Description
• Distance between two points
Interface Definition
•BOOL API_PointDist(JTRHD handle,const TcpPosition *p_from,const TcpPosition* p_to,float& distance);
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle，input
p_from	Yes	TcpPosition*	Cartesian spatial pose(unit:x/y/z (m),rx/ry/rz unit(rad))，input
p_to	Yes	TcpPosition*	Cartesian spatial pose(unit:x/y/z (m),rx/ry/rz unit(rad))，input
distance	Yes	float&	calculation result(unit:m)，input/output

Remark
typedef struct
{
double robotTcp[6]; //TThe position and posture of the end effector of the robotic arm,Corresponding to x, y, z, rx, ry, rz respectively
}TcpPosition;

4.76.Pose addition

Brief Description
•
Interface Definition
•BOOL API_PoseAdd(JTRHD handle,const TcpPosition *p_from,const TcpPosition* p_to,TcpPosition* result);
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle，input
p_from	Yes	TcpPosition*	Cartesian spatial pose(unit:x/y/z (m),rx/ry/rz unit(rad))，input
p_to	Yes	TcpPosition*	Cartesian spatial pose(unit:x/y/z (m),rx/ry/rz unit(rad))，input
result	Yes	TcpPosition*	calculation result(unit:x/y/z (m),rx/ry/rz (rad))，input/output

Remark

typedef struct
{
double robotTcp[6]; //The position and posture of the end effector of the robotic arm,Corresponding to x, y, z, rx, ry, rz respectively
}TcpPosition;

4.77.Get the inverse of a pose

Brief Description

•

Interface Definition

•BOOL API_PoseInv(JTRHD handle,const TcpPosition *p_from,TcpPosition* result);

Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle，input
p_from	Yes	TcpPosition*	Cartesian spatial pose(unit:x/y/z (m),rx/ry/rz unit(rad))，input
result	Yes	TcpPosition*	calculation result(unit:x/y/z (m),rx/ry/rz (rad))，input/output

Remark

typedef struct
{
double robotTcp[6]; //The position and posture of the end effector of the robotic arm,Corresponding to x, y, z, rx, ry, rz respectively
}TcpPosition;

4.78.Pose subtraction

Brief Description

•

Interface Definition

•BOOL API_PoseSub(JTRHD handle,const TcpPosition *p_to,const TcpPosition* p_from,TcpPosition* result);

Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle，input
p_to	Yes	TcpPosition*	Cartesian spatial pose(unit:x/y/z (m),rx/ry/rz unit(rad))，input
p_from	Yes	TcpPosition*	Cartesian spatial pose(unit:x/y/z (m),rx/ry/rz unit(rad))，input
result	Yes	TcpPosition*	calculation result(unit:x/y/z (m),rx/ry/rz (rad))，input/output

Remark

Pay attention to the order of posture before and after

typedef struct
{
double robotTcp[6]; //The position and posture of the end effector of the robotic arm,Corresponding to x, y, z, rx, ry, rz respectively
}TcpPosition;

4.79.Pose transformation

Brief Description

•

Interface Definition

•BOOL API_PoseTrans(JTRHD handle,const TcpPosition *p_from,const TcpPosition* p_from_to,TcpPosition* result);

Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle，input
p_from	Yes	TcpPosition*	Cartesian spatial pose(unit:x/y/z (m),rx/ry/rz unit(rad))，input
p_from_to	Yes	TcpPosition*	Cartesian spatial pose(unit:x/y/z (m),rx/ry/rz unit(rad))，input
result	Yes	TcpPosition*	calculation result(unit:x/y/z (m),rx/ry/rz (rad))，input/output

Remark

typedef struct
{
double robotTcp[6]; //The position and posture of the end effector of the robotic arm,Corresponding to x, y, z, rx, ry, rz respectively
}TcpPosition;

4.80.Get the radian of the robot’s zero-position joint

Brief Description

•

Interface Definition

•BOOL API_GetRobotHomeJointRad(JTRHD handle,JointRad* joint_rad);

Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle，input
joint_rad	Yes	JointRad*	input/output

typedef struct
{
double robotRad[6]; //Joint radian value
}JointRad;

Remark

•

4.81.Robot error reset

Brief Description

• Robot error reset

Interface Definition

•BOOL API_ResetRobotError(JTRHD handle);
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle，input

Remark
•

4.82.Get the robot’s runtime

Brief Description
•
Interface Definition
•BOOL API_GetRobotRunTime(JTRHD handle,long& run_time);
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle，input
run_time	Yes	int	input/output

Remark
• run_time unit: seconds, time from the start of operation to the present

4.83.Get the power-off delay time of the robot

Brief Description
•
Interface Definition
•BOOL API_GetPowerOffDelayTime(JTRHD handle,int& delay_time);
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle，input
delay_time	Yes	int	input/output

Remark
• delay_time unit: milliseconds, 0 indicates no delay

4.84.Emergency stop of the robot

Brief Description
•
Interface Definition
•BOOL API_EmergencyStop(JTRHD handle);
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle，input

Remark
•

4.85.Clear unfinished trajectory

Brief Description
• Used in scenarios where emergency stops are required and the continuation of the remaining trajectory is not desired
Interface Definition
•BOOL API_ClearUnfinishedTrajectory(JTRHD handle);
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle，input

Remark
•

4.86.Get waypoints information in the program

Brief Description
• Get waypoints information in the program，Up to 8 fixed position waypoints can be obtained.
Interface Definition
BOOL API_GetPointsInProgram(JTRHD handle,const char *program_file_name,PointsInProgram* points);
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle，input
program_file_name	Yes	char*	program name，input
points	Yes	PointsInProgram*	Description of point information，input/output.See remarks

Remark
typedef struct
{
char description[1000]; //The tool pose and joint radian of multiple points separated by ‘;’, with a maximum of 8 waypoints obtained. Each point is in the format of x, y, z, rx, ry, rz joint1, joint2, joint3, joint4, joint5, joint6
}PointsInProgram;

4.87.Get firmware version

Brief Description
• Get firmware version
Interface Definition
BOOL API_GetFirmwareVersion(JTRHD handle,VersionInfo* firmware_version);
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle，input
firmware_version	Yes	VersionInfo*	Firmware version information description, input, output, see remarks

Remark
typedef struct
{

char description[64];
}VersionInfo;

4.88.Set the installation method

Brief Description
• Set the installation method
Interface Definition
BOOL API_SetMountedPose(JTRHD handle,const MountedPose mounted_type);
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle，input
mounted_type	Yes	MountedPose	Installation method type, input, see remarks

Remark
typedef enum {
FLOOR_MOUNTED = 0,
SIDE_MOUNTED,
INVERTED_MOUNTED,
}MountedPose;

4.89.Set the installation angle

Brief Description
• Set the tilt angles and rotation angles of the base
Interface Definition
BOOL API_SetMountedAngle(JTRHD handle,const float tiltAngle,const float baseRotationAngle);
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle，input
tiltAngle	Yes	float	Install tilt angle, input
baseRotationAngle	Yes	float	Base rotation angle, input

Remark
•

4.90.End tool Modbus RTU read and write

Brief Description
• End tool Modbus RTU read and write
Interface Definition
BOOL API_ModbusRtuRWEndTool(JTRHD handle,const ModbusRTU_Request *req,ModbusRTU_Frame* rsp,int timeout=2);
Parameter

Parameter Name	Required	Type	Description
handle	Yes	JTRHD	Login handle，input
req	Yes	ModbusRTU_Request*	request cmd
rsp	Yes	ModbusRTU_Frame*	response cmd

Parameter Name	Required	Type	Description
timeout	Yes	int	timeout: default 2s

Remark
typedef struct
{
unsigned char slave_addr;
unsigned char func_code;
unsigned short start_addr;
unsigned short data_number; // Data quantity (number of registers/coils)
unsigned char data[250]; // The message data (up to 255 bytes) is binary data, with each byte ranging from 0x00 to 0xFF
unsigned char data_len; //Actual message length, actual number of bytes in binary data
}ModbusRTU_Request;

typedef struct {
unsigned char data[256]; // The message data (up to 255 bytes) is binary data, with each byte ranging from 0x00 to 0xFF
unsigned short data_len; // Actual message length, actual number of bytes in binary data
} ModbusRTU_Frame;